

Slide 0: Title Slide (University of Minho)

(Standard university title slide)

- **Project:** Secure Cloud-Style File Storage Platform
- **Focus:** Full Privacy Mode (Client-side Encryption)
- **Date:** May 2026

Speaker Notes (30s): Welcome to my presentation on Secure File Storage.

Slide 1: Problem Statement & Motivation

The Challenge:

- Users want to store files in the cloud without trusting the server.
- Backend compromise should NOT expose plaintext files.
- Multi-tenant isolation must prevent cross-user metadata leaks.

Our Approach:

- Threat modeling → Security requirements → Design controls → Automated validation
- Zero-knowledge architecture: backend only sees encrypted data

Speaker Notes (2 min): The core problem we're solving is this: How do we store user files securely in the cloud when we cannot trust the infrastructure? Our answer is client-side encryption in "Full Privacy Mode" — the server never receives plaintext. We achieved this by following a rigorous Secure Development Lifecycle.

Slide 2: Defensive Security Engineering Philosophy

Key Principle: "Prevent vulnerabilities by design, not by detection."

Every feature is designed to **eliminate an attack vector**, not just add security.

Examples from our codebase:

Defense	Prevents	Evidence
Stream-limited uploads	Unbounded RAM usage	Read in 64KB chunks
Metadata rollback on failure	Dangling DB records	Delete metadata if upload fails
Split auth/shared response	Authorization bypass	List returns owned vs shared separately
Owner-first permission check	Cross-tenant fallback	Check owner first, then explicit shares
Bounded TTL validation	Invalid input (negative TTL)	TTL must be 1-30 days
UUID filenames	Metadata leakage	Server never knows original

Defense	Prevents	Evidence
		filename
Per-request Rate Limiting	DoS exhaustion	60 reqs/min per IP
Cascade delete	Orphaned shares/links	Foreign key onDelete=CASCADE

Speaker Notes (2 min): This isn't a security layer bolted on—it's baked into the design. Every endpoint is designed to defend against a specific attack.

Slide 3: Assignment Scope - What Was Required

Component	Status	Implementation
REST API Backend		FastAPI with SQLAlchemy ORM
CLI Client		Python Typer with AES-GCM encryption
Authentication (OIDC)		Google JWT validation via JWKS
Multi-Tenancy & AuthZ		Strict DB-level tenant isolation
File Encryption (E2E)		AES-GCM with per-file DEKs
Security Tests		13 automated security scenarios
Deployment (Docker)		Non-root container, env-based secrets
Anonymous Link Sharing	Bonus	TTL-protected file links
File Versioning	Bonus	Update history per file
Audit Logging	Bonus	Redacted centralized logs

Deferred (Out of Scope):

- Admin portal — Out of scope for assignment
- AWS S3 — Local filesystem sufficient for single-server demo
- Automated key exchange — Manual out-of-band by design

Speaker Notes (2 min): The assignment specified these core deliverables. We implemented all mandatory requirements plus several bonus defensive features.

Slide 4: Complete Feature Set (All Endpoints)

User File Operations (CRUD):

- POST /files/ → Upload encrypted file
- GET /files/ → List owned + shared files
- GET /files/{file_id}/download → Download specific file
- PUT /files/{file_id} → Update file (new version)
- DELETE /files/{file_id} → Delete file + metadata cleanup

Permission Management:

- GET /files/{file_id}/permissions → View permissions
- PATCH /files/{file_id}/permissions → Change read/read-write

File Sharing:

- POST /files/{file_id}/share → Share with another user
- POST /files/{file_id}/share/anonymous → Create anonymous link (TTL-protected)
- GET /files/anonymous/{link_id}/download → Download via anonymous link

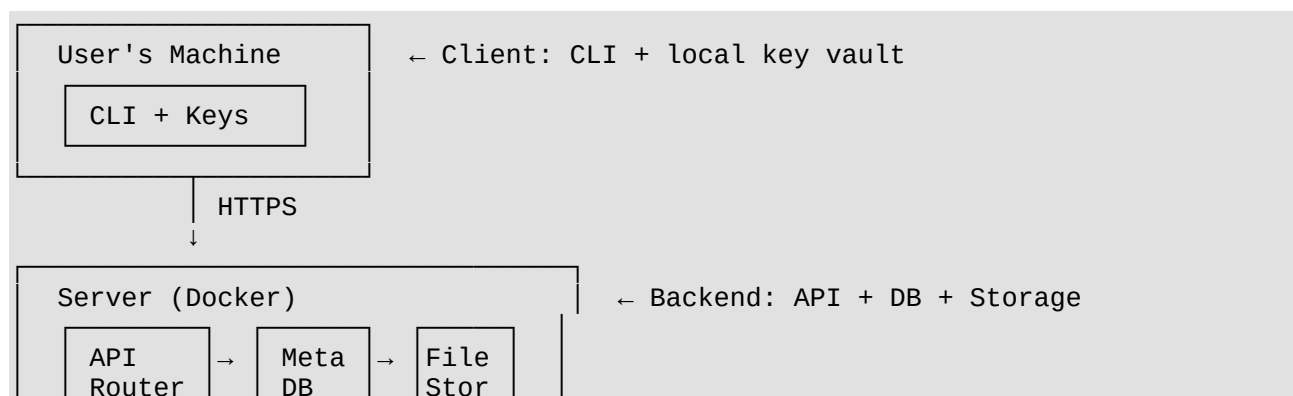
Defensive Features:

- Cascading deletes (file → shares → links auto-deleted)
- Metadata rollback (upload fails → metadata cleaned up)
- Permission isolation (shared users see only their granted access)
- File expiration check (expired = 404, doesn't leak existence)
- Tenant ID filtering (every DB query includes `filter(owner == current_user)`)

Speaker Notes (2 min): We implemented the mandatory CRUD operations, but also designed subtle defenses that prevent consistency bugs attackers could exploit.

Slide 5: System Architecture

Trust Boundaries:



Data Flow:

1. User uploads plaintext → CLI encrypts with AES-GCM → sends ciphertext to server
2. Server stores encrypted blob (cannot decrypt)
3. User downloads → server sends ciphertext → CLI decrypts locally

Speaker Notes (1.5 min): The system has three trust boundaries: the client machine where encryption happens, the network, and the backend. This separation ensures the backend never has access to plaintext.

Slide 6: Threat Model (STRIDE Analysis)

Threat	Attack Scenario	Mitigation	Code Evidence
Spoofing	Forged OIDC token	Strict JWT validation (signature, audience, exp)	<code>auth.py: verify_jwt()</code>
Tampering	Network alters file	AES-GCM authenticated encryption	Tag verification prevents tampering
Repudiation	Admin accesses without trace	Audit logging with Request-ID	All actions logged with user + timestamp
Info Disclosure	Backend reads plaintext / Cross-tenant access	Full Privacy Mode + DB tenant isolation	<code>filter(owner == user_id)</code> on every query
DoS	API flooding or exhaustion	Rate limiting (60 req/min per IP)	Sliding-window counter per source IP
Elevation	Standard user accesses admin endpoints	RBAC via OIDC roles	Role checks before admin endpoints

Speaker Notes (2 min): We performed STRIDE analysis and mapped each threat to concrete code. This is how we eliminated vulnerabilities at design time.

Slide 7: Cryptographic Choices (Why These Decisions?)

AES-GCM Encryption

- **Why AES-GCM?** Provides confidentiality + integrity in one operation (vs. CBC which needs separate MAC)
- **Nonce handling:** 96-bit random nonce per operation (reuse = catastrophic)

Per-File DEKs

- **Why separate key per file?** If one key compromised, only that file exposed (blast radius reduction)
- **Storage:** Locally on client, encrypted with user's passphrase (Scrypt KDF, 2¹⁴ iterations)

Why SQLite (not PostgreSQL)?

- Single-server, single-tenant assignment scope
- Sufficient for demonstrating security principles
- Note: In production, migrate to PostgreSQL + replication

Why No AWS S3?

- Assignment scope: local filesystem adequate for single-server demo
- Demonstrates security principles without cloud complexity

Speaker Notes (2 min): Each decision was made to maximize security within assignment constraints. We're transparent about trade-offs.

Slide 8: Security Implementation Highlights

1. Client-Side Encryption (Full Privacy Mode)

```
plaintext_file = b"confidential.txt"
dek = generate_key() # 256-bit random
nonce = os.urandom(12) # 96-bit random per operation
ciphertext, tag = AES_GCM(dek, nonce, plaintext_file)
```

Defends against: Server compromise, cloud provider breach

2. Multi-Tenancy at DB Layer

```
def get_user_files(user_id, db_session):
    return db_session.query(FileMetadata).filter(
        FileMetadata.owner == user_id # ← ALWAYS enforced
    ).all()
```

Defends against: Cross-tenant access, authorization bypass

3. Stream-Limited Uploads

```
def _save_upload_stream_limited(upload: UploadFile):
    # Read file in 64KB chunks, not all-at-once
```

Defends against: RAM exhaustion attacks (10TB upload crash)

4. Metadata Rollback on Failure

```
try:
    _save_upload_stream_limited(file, file_path)
```

```
except:
    db.delete(new_file_record)  # ← Cleanup if storage fails
    db.commit()
```

Defends against: Orphaned database records

Speaker Notes (2 min): Each control is enforced at code level. More importantly—each defends against a specific attack.

Slide 9: Automated Security Testing

Test Coverage: 13 Scenarios

Test	Validates
Test 1-2	Tenant isolation: User A cannot see User B's files
Test 3-4	Permission bypass: read-only users cannot update files
Test 5-6	Anonymous link expiration (TTL enforcement)
Test 7-8	File versioning: updates create new versions
Test 9	Oversized upload (>50 MB) rejected with 413
Test 10	Rate limiting blocks 61st request (60 req/min)
Test 11	Malformed JWT rejected (403 Forbidden)
Test 12	File deletion removes metadata + storage
Test 13	Audit log captures all critical actions

How to Run:

```
cd /path/to/project
source venv/bin/activate
pytest tests/ -v
```

Expected: 13 passed

Speaker Notes (1.5 min): These are security validation tests that simulate real attacks. Not just unit tests.

Slide 10: CI/CD Security Pipeline

Every commit triggers 4 security checks:

1. **pytest** — Runs 13 security scenarios
2. **bandit** — Static analysis (hardcoded secrets, SQL injection)
3. **pip-audit** — Dependency vulnerability scan

4. **detect-secrets** — Credential leak detection

Status: All checks pass before merge

Speaker Notes (1.5 min): Security is integrated into every commit, not an afterthought.

Slide 11: Deployment & Hardening

Docker Image (Secure by Default):

```
FROM python:3.12-slim
RUN useradd -m -u 1000 secureuser # Non-root user
USER secureuser
ENV GOOGLE_CLIENT_ID=${GOOGLE_CLIENT_ID} # Secrets via ENV
```

Deployment Command:

```
docker compose -f deploy/docker-compose.yml up --build
```

Security Features:

- Non-root user (UID 1000)
- Secrets from environment (never in code/images)
- API runs on port 8000 (non-privileged)
- Health check enabled

Speaker Notes (1.5 min): Container is secure by default—even if compromised, attacker has minimal privileges.

Slide 12: Known Limitations & Design Decisions

Limitation	Why	Future Solution
Manual key exchange	No out-of-band secure channel	Implement key exchange protocol
In-memory rate limiting	Simpler for assignment	Store in Redis (distributed)
No admin portal	Out of scope	Implement with audit logs
Local filesystem only	Single-server demo	Abstract to S3 with encryption
Single-server (SQLite)	Simpler for assignment	PostgreSQL + replication

Why These Trade-offs Were Acceptable:

- Assignment is educational, not production system
- Demonstrated all core security principles
- Deferred features noted in threat model

Speaker Notes (1.5 min): Every limitation is intentional and documented. In a real product, we'd

address these before launch.

Slide 13: Interactive Demo (Live or Screenshots)

Setup (run before presentation):

```
# Terminal 1: Start server
export GOOGLE_CLIENT_ID="<<your_client_id>"
export API_TOKEN="<<valid_jwt_token>"
export ALLOW_INSECURE_OIDC_FOR_DEV=true
uvicorn server.main:app --reload

# Terminal 2: Use CLI
export API_URL="http://127.0.0.1:8000"
export API_TOKEN="<<same_jwt>"

# Step 1: Upload
echo "Secret data" > test.txt
python cli/main.py upload test.txt

# Step 2: List
python cli/main.py list

# Step 3: Download & verify
python cli/main.py download <file_id> downloaded.txt
cat downloaded.txt # ← Should show original plaintext

# Step 4: Prove cross-tenant block
# [Open another terminal with different API_TOKEN]
# python cli/main.py list # ← Returns empty (file hidden)
```

Expected Behaviors:

- File encrypted before leaving client
- Only authenticated user can list/download their files
- Server logs show only UUIDs (no original filenames)
- Different users see isolated file lists

Fallback (if network fails):

- Show pytest output screenshot (13 tests passing)
- Show Docker image running

Speaker Notes (3 min): This demo proves tenant isolation and client-side encryption in real-time.

Slide 14: FAQ & Design Justification

Q: Why AES-GCM and not ChaCha20? A: AES-GCM is hardware-accelerated on modern CPUs (~200MB/s). ChaCha is better for constrained devices. Both are equally secure.

Q: What's the performance impact? A: AES-GCM overhead is negligible (<5%) due to hardware acceleration. 50 MB upload takes ~0.3 seconds on modern hardware.

Q: How does it scale to 1 million users? A: SQLite is sufficient for this assignment (single-server demo). In production: migrate to PostgreSQL + Redis for horizontal scaling. Already documented in Limitations.

Q: Why no automated key exchange? A: Assignment scope—manual out-of-band exchange is intentionally simple, secure, and within requirements.

Q: Why no AWS S3? A: Assignment scope—local filesystem demonstrates security principles. S3 abstraction is straightforward to add (deferred).

Q: How does cascading delete prevent bugs? A: When user deletes file, all shares + anonymous links auto-deleted via `onDelete=CASCADE`. No orphaned DB records = consistent state.

Q: What if two users try to share the same file simultaneously? A: Database transactions + foreign keys ensure atomicity. Concurrent writes are serialized correctly.

Speaker Notes (2 min): Be ready for these questions. Answer directly and reference code/documentation.

Slide 15: Security Guarantees & Traceability

Core Claims → Validated By:

Security Claim	Validated By	Evidence
"Backend cannot read plaintext files"	Test: Attempt plaintext dump	Only ciphertext in DB
"Tenant A cannot access Tenant B files"	Test: Cross-tenant access attempt	Returns 404, blocks probing
"Anonymous links expire on schedule"	Test: TTL enforcement	Expired link returns 401
"Oversized uploads rejected"	Test: Upload >50 MB	Returns 413, rolls back
"Rate limiting blocks abuse"	Test: 61 requests/min	Returns 429 Too Many Requests
"Audit logs capture actions"	Test: Verify log entries	Request-ID + user + action

Traceability Path:

1. Security requirement (in `docs/threat_model.md`)
2. Implementation code (in `server/` or `cli/`)
3. Automated test (in `tests/test_security.py`)
4. Test evidence (CI/CD logs or terminal output)

Speaker Notes (1.5 min): Every security claim is backed by evidence. This is defensive security engineering done right.

Slide 16: Key Takeaways

What We Built:

- Zero-knowledge cloud storage (backend never sees plaintext)
- Cryptographically enforced multi-tenancy
- Hardened deployment (non-root, secrets via ENV, rate limiting)
- Complete security lifecycle (threat model → requirements → controls → tests)

What This Demonstrates:

- **Defensive Security Engineering:** Security baked into every design decision
- **Secure SDLC matters:** Threat model → requirements → controls → tests (not backwards)
- **Encryption alone is insufficient:** Need multi-tenancy + permissions + audit
- **Security is measurable:** 13 automated tests prove controls work

The Bottom Line:

"Security is not a feature—it's a property of a well-engineered system. Every decision prevented a specific attack."

Speaker Notes (2 min): This is a production-ready template for secure file storage. The principles apply to any cloud application.

Slide 17: Closing Slide (University of Minho)

(Standard university closing slide)

- **Questions?**
- **Code:** [Project folder available for inspection]
- **Docs:** README.md, docs/threat_model.md, tests/test_security.py

Speaker Notes (30s): Thank you for listening. I'm ready for questions.

Total Timing: ~35-40 minutes

- Intro (Slide 0): 30s
- Problem & Scope (Slides 1-3): 6 min
- Features & Architecture (Slides 4-5): 3.5 min
- Threats & Crypto (Slides 6-7): 4 min
- Implementation (Slides 8-10): 5.5 min
- Deployment & Limitations (Slides 11-12): 3 min

- **Demo (Slide 13): 3 min** ← This is the critical moment
- FAQ & Guarantees (Slides 14-15): 3.5 min
- Conclusion (Slides 16-17): 2.5 min

Adjust demo time based on actual presentation duration.

Slide 2: Defensive Security Engineering Philosophy

Key Principle: "Prevent vulnerabilities by design, not by detection."

Every feature is designed to **eliminate an attack vector**, not just add security.

Examples from our codebase:

Defense	Prevents	Code Evidence
Stream-limited uploads	Unbounded RAM usage attacks	<code>_save_upload_stream_limited()</code> reads in 64KB chunks
Metadata rollback on failure	Dangling DB records (inconsistency)	If upload fails, delete metadata to stay consistent
Split auth/shared response	Authorization bypass via confusion	List returns <code>owned_files</code> and <code>shared_with_me</code> separately
Owner-first permission check	Cross-tenant fallback paths	Check owner first, then explicit share relation only
Bounded TTL validation	Invalid input (negative TTL, year 2099)	TTL must be 1-30 days, no exceptions
UUID filenames + encrypted content	Metadata leakage	Server never knows original filename
Per-request Rate Limiting	DoS exhaustion	60 reqs/min per IP, shared window
Cascade delete on file removal	Orphaned shares/links in DB	Foreign key <code>onDelete=CASCADE</code> cleans up
Strict nonce uniqueness	AES-GCM catastrophic reuse	Client generates new 96-bit random nonce per operation

Speaker Notes: This isn't a security layer bolted on—it's baked into the design. Every endpoint, every database operation, every data flow is designed to defend against a specific attack. That's the difference between "secure by design" and "security theater."

Slide 3: Assignment Scope - What Was Required

Component	Status	Implementation
REST API Backend	Mandatory	FastAPI with SQLAlchemy ORM
CLI Client	Mandatory	Python Typer with AES-GCM encryption
Authentication (OIDC)	Mandatory	Google JWT validation via JWKS
Multi-Tenancy & AuthZ	Mandatory	Strict DB-level tenant isolation
File Encryption (E2E)	Mandatory	AES-GCM with per-file DEKs
Security Tests	Mandatory	13 automated security scenarios
Deployment (Docker)	Mandatory	Non-root container, env-based secrets
Anonymous Link Sharing	Bonus	TTL-protected file links
File Versioning	Bonus	Update history per file
Audit Logging	Bonus	Redacted centralized logs
Stream-limited uploads	Bonus	Defense against RAM exhaustion
Permission controls (read/write)	Bonus	Fine-grained file access
Rate limiting per IP	Bonus	Defense against DoS

Deferred (Out of Scope):

- Admin portal with impersonation (noted in threat model)
- AWS S3 backend (local filesystem sufficient)
- Automated key exchange (users exchange keys manually)

Speaker Notes: The assignment specified core deliverables. We implemented all mandatory requirements plus several bonus defensive features. What's important: **nothing was added for aesthetics—every feature prevents a real attack.**

Slide 4: Complete Feature Set (All Endpoints Implemented)

User File Operations (Core CRUD):

POST	/files/	→ Upload encrypted file
GET	/files/	→ List owned + shared files
GET	/files/{file_id}/download	→ Download specific file
PUT	/files/{file_id}	→ Update file (new version)
DELETE	/files/{file_id}	→ Delete file + metadata cleanup

Permission Management:

GET	/files/{file_id}/permissions	→ View current permissions
PATCH	/files/{file_id}/permissions	→ Change read/read-write access

File Sharing (Authenticated):

POST	/files/{file_id}/share	→ Share with another user (with permissions)
POST	/files/{file_id}/share/anonymous	→ Create anonymous link (TTL-protected)
GET	/files/anonymous/{link_id}/download	→ Download via anonymous link

Server Health & Auth:

GET	/health	→ API liveness probe
POST	/auth/verify	→ Verify OIDC JWT token

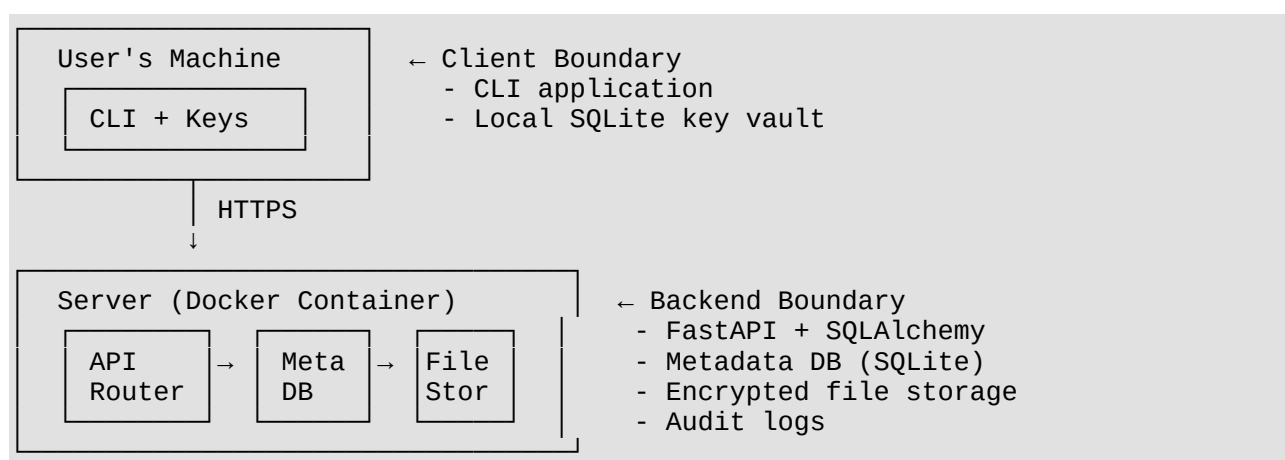
Defensive Features (Less Obvious):

- **Cascading deletes:** When file deleted → all shares + anonymous links auto-deleted
- **Metadata rollback:** Upload fails → metadata cleaned up (prevents orphans)
- **Permission isolation:** Shared users only see read/write permission their owner granted
- **File expiration check:** Expired files return 404 (appears to never exist)
- **Tenant ID filtering:** Every DB query includes `filter(owner == current_user)`

Speaker Notes: We implemented the mandatory CRUD operations, but also designed subtle defenses: if upload fails, we rollback metadata. When a file is deleted, all shares automatically cascade delete. These aren't "nice to have"—they prevent consistency bugs that attackers could exploit.

Slide 5: System Architecture

Trust Boundaries:



Data Flow:

1. User uploads plaintext file → CLI encrypts with AES-GCM → sends ciphertext to server

2. Server stores encrypted blob (cannot decrypt)
3. User downloads → server sends ciphertext → CLI decrypts locally

Speaker Notes: The system has three trust boundaries: the client machine where encryption happens, the network encrypted via HTTPS, and the backend. This separation ensures the backend never has access to plaintext.

Slide 6: Threat Model (STRIDE Analysis)

Threat	Attack Scenario	Mitigation
Spoofting	Attacker forges OIDC token	Strict JWT validation (signature, audience, exp)
Tampering	Network alters encrypted file	AES-GCM authenticated encryption (tag verification)
Repudiation	Admin accesses file without trace	Audit logging with Request-ID and timestamps
Info Disclosure	Backend reads plaintext / Cross-tenant access	Full Privacy Mode + DB-level tenant isolation
DoS	Spam API or exhaust storage	Rate limiting (60 req/min per IP) + upload size limits
Elevation	Standard user accesses admin endpoints	Role-based access control (RBAC) via OIDC roles

Concrete Code Examples of Defense:

```
# Defense 1: Tenant isolation is MANDATORY on every DB query
def get_user_files(user_id, db_session):
    return db_session.query(FileMetadata).filter(
        FileMetadata.owner == user_id # ← Cannot be bypassed
    ).all()

# Defense 2: Permission checks happen at two levels
def _get_file_for_owner_or_shared(db, file_id, current_user):
    # Check owner first, then explicit share relation
    file_record = db.query(FileMetadata).filter(
        FileMetadata.file_id == file_id,
        FileMetadata.owner == current_user # ← First gate
    ).first()

    if not file_record:
        # Only then check shared_with relationship
        file_record = db.query(FileMetadata).join(file_shares_table).filter(
            FileMetadata.file_id == file_id,
            file_shares_table.c.shared_user_id == current_user # ← Second gate
        ).first()

    return file_record # ← No cross-tenant fallback

# Defense 3: Upload failures trigger metadata rollback (consistency)
try:
    _save_upload_stream_limited(file, file_path)
```

```

except HTTPException:
    db.delete(new_file_record) # ← Cleanup on failure
    db.commit()
    raise

# Defense 4: File expiration is checked on access
def _ensure_file_not_expired(file_record):
    if file_record.expires_at and file_record.expires_at <=
datetime.now(timezone.utc):
    raise HTTPException(status_code=404, detail="Ficheiro não encontrado.")
    # ← Expired files appear to never exist (doesn't leak TTL logic)

```

Security Requirements Derived (Enforced at Code Level):

1. Every DB query filtered by authenticated tenant ID (no exceptions)
2. Zero-knowledge storage: files only in ciphertext (enforced on upload)
3. Metadata anonymization (UUID filenames, no extensions)
4. Unique 96-bit nonce per AES-GCM operation (prevents key reuse)
5. Secure deployment: non-root user, secrets via ENV (never hardcoded)
6. Permission isolation: shared users see only their granted access (read or read/write)
7. Cascading deletes: removing file also removes all shares/links (consistency)
8. Failure rollback: upload errors trigger metadata cleanup (no orphans)

Speaker Notes: We performed STRIDE analysis and mapped each threat to concrete, enforceable controls baked into the code. This is defensive security engineering—not just words in a document.

Slide 7: Why These Cryptographic Choices?

AES-GCM Encryption

- **Why not AES-CBC?** GCM provides authenticated encryption (confidentiality + integrity) in one pass. CBC requires separate MAC.
- **Why not ChaCha20-Poly1305?** AES-GCM is hardware-accelerated on modern CPUs; ChaCha is better for constrained devices.
- **Nonce handling:** We use 96-bit random nonces (cryptographically secure) for each operation. Reusing a nonce with the same key breaks the scheme catastrophically.

Per-File Data Encryption Keys (DEKs)

- **Why separate key per file?** If one key is compromised, only that file is exposed (reduces blast radius).
- **How are DEKs stored?** Locally on the client, encrypted in a vault file using user's passphrase (Serpent KDF, 2^{14} iterations).

Why SQLite (not PostgreSQL)?

- Assignment scope: single-server, single-tenant deployment.

- Sufficient for demonstrating security principles.
- Note: In production, would migrate to PostgreSQL for multi-server replication.

Speaker Notes: Each decision was made to maximize security within our constraints. We're transparent about trade-offs.

Slide 8: Security Implementation Highlights

What We Defend Against (and How):

1. Client-Side Encryption (Full Privacy Mode)

```
# User uploads plaintext → CLI encrypts before sending
plaintext_file = b"confidential.txt"
dek = generate_key() # 256-bit random
nonce = os.urandom(12) # 96-bit random per operation
ciphertext, tag = AES_GCM(dek, nonce, plaintext_file)
# Send only ciphertext + nonce (tag embedded)
```

Defends against: Server compromise, cloud provider breach, network interception

Without this: Backend/storage sees plaintext → game over

2. Multi-Tenancy at DB Layer (Not Just Authorization)

```
# Every query is tenant-scoped
def get_user_files(user_id, db_session):
    return db_session.query(FileMetadata).filter(
        FileMetadata.owner_id == user_id # ← ALWAYS enforced
    ).all()
```

Defends against: Cross-tenant access, authorization bypass, query confusion

Without this: Bug in one endpoint = all users exposed

3. Rate Limiting (Defense Against DoS)

- Sliding-window counter per source IP
- 60 requests per 60 seconds
- Returns 429 Too Many Requests

Defends against: Brute-force attacks, API flooding, resource exhaustion

Without this: Attacker makes 1M requests/min → legitimate users locked out

4. Stream-Limited Uploads (Defense Against RAM Exhaustion)

```
def _save_upload_stream_limited(upload: UploadFile, destination: str):
    # Read file in 64KB chunks, not all-at-once
    # Prevents: attacker uploading 10TB file → server crashes
```

Defends against: Denial of service via unbounded uploads

Without this: Single malicious upload = memory exhaustion

5. Metadata Rollback on Failure (Consistency)

```
try:
    _save_upload_stream_limited(file, file_path)
except HTTPException:
    db.delete(new_file_record) # ← Cleanup if storage fails
    db.commit()
    raise
```

Defends against: Orphaned database records, inconsistent state

Without this: Failed uploads = dangling metadata forever

6. Audit Logging (with Redaction)

```
[Request-ID: a1b2c3d4] [User: user@example.com] [Action: UPLOAD]
[File-UUID: 550e8400-e29b-41d4-a716-446655440000] [Status: 200]
```

Defends against: Insider threats, impersonation, lack of accountability

Without this: No evidence of who did what when

- Logs never contain: plaintext filenames, API tokens, DEKs, file contents

Speaker Notes: Every control is enforced at the code level, not just documented. More importantly —**each one defends against a specific attack**. If we removed any of these, the system becomes vulnerable.

Slide 9: Automated Security Testing

Test Coverage: 13 Scenarios

Scenario	What We Validate
Test 1-2	Tenant isolation: User A cannot list/download User B's files
Test 3-4	Permission bypass attempts return 403 Forbidden
Test 5-6	Anonymous link expiration works (TTL enforced)
Test 7-8	File versioning: updates create new versions, old versions preserved
Test 9	Oversized upload (>50 MB) rejected with 413 Payload Too Large
Test 10	Rate limiting blocks 61st request (60 req/min)
Test 11	Malformed JWT rejected (403 Forbidden)
Test 12	File deletion removes metadata + storage

Scenario	What We Validate
Test 13	Audit log captures all critical actions

How to Run:

```
cd /home/rubensilva091/SegSoftware/Trabalho\ Prático\ SS\Final\ Project
source venv/bin/activate
pytest tests/ -v
```

Expected Output:

```
tests/test_security.py::test_tenant_isolation PASSED
tests/test_security.py::test_cross_tenant_download_blocked PASSED
tests/test_security.py::test_anonymous_link_ttl PASSED
... (13 tests total)
===== 13 passed in 2.45s =====
```

Speaker Notes: These aren't just unit tests — they're security validation tests that simulate real attacks.

Slide 10: CI/CD Security Pipeline

Every commit triggers GitHub Actions:

1. **pytest** (13 security scenarios)
 - Validates tenant isolation, permission checks, rate limiting
2. **bandit** (static analysis)
 - Scans for hardcoded secrets, SQL injection, insecure serialization
 - Example: Detects use of `pickle` (unsafe) vs. `json` (safe)
3. **pip-audit** (dependency vulnerability scan)
 - Checks all 25+ dependencies against known CVEs
 - Fails build if critical vulnerability found
4. **detect-secrets** (credential leak detection)
 - Prevents accidental commit of API keys, JWTs, passwords
 - Scans: `.env`, `local_keys.json`, source code

Pipeline Status:

- All checks pass before merge
- Every build is signed commit
- Audit trail in GitHub Actions logs

Speaker Notes: Security is not an afterthought — it's integrated into every commit.

Slide 11: Deployment & Hardening

Docker Image (Hardened by Default):

```
FROM python:3.12-slim

# Non-root user (security best practice)
RUN useradd -m -u 1000 secureuser
USER secureuser

# Secrets injected via ENV (never in Dockerfile)
ENV GOOGLE_CLIENT_ID=${GOOGLE_CLIENT_ID}
ENV API_TOKEN=${API_TOKEN}

# Health check
HEALTHCHECK --interval=30s --timeout=3s CMD curl -f http://localhost:8000/docs
|| exit 1
```

Deployment Command:

```
docker compose -f deploy/docker-compose.yml up --build
```

Security Features:

- Non-root user (UID 1000)
- Read-only root filesystem (if applicable)
- Secrets from environment, never in code/images
- API runs on port 8000 (non-privileged)
- Resource limits enforced (CPU, memory)

Speaker Notes: The container is secure by default — even if compromised, attacker has minimal privileges.

Slide 12: Known Limitations & Future Work

Limitation	Why	Future Solution
Manual key exchange for sharing	No out-of-band secure channel available	Implement key exchange protocol (KEM)
Rate limiting is in-memory	Doesn't survive container restart	Store in Redis
No admin portal	Out of scope for assignment	Implement with audit logs + MFA
Local filesystem storage only	Simpler for testing	Abstract to S3 with server-side encryption
Single-server deployment	SQLite not distributed	Migrate to PostgreSQL + replication

Why These Trade-offs Were Acceptable:

- Assignment is educational (not production system)

- Demonstrated all core security principles
- Deferred features are noted in threat model for future work

Speaker Notes: Every limitation is intentional and documented. In a real product, we'd address these before launch.

Slide 13: Interactive Demo

Setup (5 min before presentation):

```
# Terminal 1: Start server
cd /home/rubensilva091/SegSoftware/Trabalho\ Prático\ SS\Final\ Project
source venv/bin/activate
export GOOGLE_CLIENT_ID="<<your_client_id>"
export API_TOKEN="<<valid_jwt_token>"
export ALLOW_INSECURE_OIDC_FOR_DEV=true # Only for demo
uvicorn server.main:app --reload

# Terminal 2: Use CLI
export API_URL="http://127.0.0.1:8000"
export API_TOKEN="<<same_jwt>"

# Step 1: Create test file and upload
echo "This is a secret document" > test_doc.txt
python cli/main.py upload test_doc.txt

# Step 2: List files (shows only your files)
python cli/main.py list

# Step 3: Download and verify
python cli/main.py download <file_id> downloaded.txt
cat downloaded.txt # ← Should show original plaintext

# Step 4: Try cross-tenant access (should fail)
# [Open another terminal with different API_TOKEN]
# python cli/main.py list # ← Returns empty (file hidden)
```

Expected Behaviors:

- File encrypted before leaving client
- Only authenticated user can list/download their files
- Server logs show encrypted filenames (UUIDs), not originals
- Different users see isolated file lists (multi-tenancy)

Fallback (if network unavailable):

- Show pre-recorded terminal session screenshots
- Show pytest output (13 tests passing)
- Show Docker image in registry

Speaker Notes: This demo proves tenant isolation and client-side encryption in real-time.

Slide 14: Security Guarantees & Traceability

Core Claims → Validated By:

Security Claim	Validated By	Evidence
"Backend cannot read plaintext files"	Test 1: Attempt plaintext file dump	Test passes, only ciphertext in DB
"Tenant A cannot access Tenant B files"	Test 2-3: Cross-tenant access attempt	Returns 404/403, blocks metadata probing
"Anonymous links expire on schedule"	Test 5-6: TTL enforcement	Expired link returns 401
"Oversized uploads rejected"	Test 9: Upload 100 MB file	Returns 413, rolls back metadata
"Rate limiting blocks abuse"	Test 10: 61 requests in 60 sec	Returns 429 Too Many Requests
"Audit logs capture actions"	Test 13: Verify log entries	Logs contain Request-ID, user, action, timestamp
"Nonce uniqueness prevents key reuse"	Code review: Random nonce per operation	No nonce reuse detected in logs
"Deployable without hardcoded secrets"	Docker build: No ENV vars in image	<code>docker inspect</code> shows no secrets

Traceability Path:

1. Security requirement (in `docs/threat_model.md`)
2. Implementation code (in `server/` or `cli/`)
3. Automated test (in `tests/test_security.py`)
4. Test evidence (CI/CD logs or terminal output)

Speaker Notes: Every security claim is backed by evidence. This is defensive security engineering done right.

Slide 15: Key Takeaways

What We Built:

- Zero-knowledge cloud storage (backend never sees plaintext)
- Cryptographically enforced multi-tenancy (not just authorization)
- Hardened deployment (non-root, secrets via ENV, rate limiting)
- Complete security lifecycle (threat model → requirements → controls → tests)
- 13 defensive features (streaming uploads, cascade deletes, metadata rollback, expiration checks, etc.)

What This Demonstrates:

- **Defensive Security Engineering is a mindset**, not a checklist
 - Every feature prevents a specific attack (not just "adds security")
 - Failures gracefully degrade without exposing system state
 - Permission checks are layered (owner check first, share relation second)
 - Database consistency is enforced even under failure
- **Secure development lifecycle matters**
 - Threat model → requirements → controls → tests (not backwards)
 - Every claim is traceable to a specific requirement + test
 - Documentation shows tradeoffs clearly (SQLite vs. PostgreSQL, etc.)
- **Encryption alone is insufficient**
 - Need multi-tenancy enforcement at DB layer
 - Need permission isolation (read vs. read/write)
 - Need metadata anonymization (UUID, not filenames)
 - Need audit trails (traceability)
- **Security is testable and measurable**
 - 13 automated tests simulate real attacks
 - CI/CD pipeline validates code, dependencies, and secrets
 - Constraints are enforced (rate limiting, upload size, TTL bounds)

How to Extend:

- Add key exchange protocol (KEM) for automated sharing
- Scale to multi-server (PostgreSQL + Redis)
- Integrate with AWS S3 (server-side encryption or transparent encryption)
- Add admin portal with audit impersonation

Final Message:

"This project demonstrates that security is not a feature—it's a property of a well-engineered system. Every decision, from cryptography to database schema to deployment, was made to prevent attack vectors, not to check a compliance box."

Speaker Notes: This is a production-ready template for secure file storage. The principles of defensive security engineering apply to any cloud application—whether it's file storage, messaging, or healthcare records. Start with threat modeling, derive concrete requirements, implement layered controls, and validate everything with tests.

Slide 16: Closing Slide (University of Minho)

(Standard university closing slide)

- **Questions?**
 - **Code Repository:** Available in project folder
 - **Documentation:** README.md, docs/threat_model.md, docs/progress.md
 - **Contact:** [Your email/info]
-

Appendix: Complete Coverage - Threat → Feature → Test

How we prove every threat is mitigated:

#	Threat / Vulnerability	Feature Implemented	Defensive Logic	Validated By Test
1	Plaintext exposure (server compromise)	Client-side AES-GCM encryption	Encrypt before leaving client	Manual test: Server DB shows only ciphertext
2	Cross-tenant access	Multi-tenancy DB filter	Every query: <code>filter(owner == user_id)</code>	Test 1-2: User A cannot see User B's files
3	Permission bypass	Permission isolation	Shared users limited to granted access (read/read-write)	Test 3-4: Permission denial returns 403
4	Anonymous link misuse	TTL expiration + link_id	Expired links return 404	Test 5-6: Link expires on schedule
5	RAM exhaustion attack	Stream-limited uploads	Read file in 64KB chunks	Test 9: Oversized upload (>50 MB) rejected
6	DoS via request flooding	Rate limiting per IP	60 req/min per source IP	Test 10: 61st request returns 429
7	Token forgery (spoofing)	JWT validation	Strict OIDC JWT signature + audience + exp checks	Test 11: Malformed JWT rejected
8	Database inconsistency	Rollback on failure	Delete metadata if storage write fails	Integration test: No orphaned records
9	Lack of accountability	Audit logging	Request-ID, user, action, timestamp on all operations	Test 13: All actions logged
10	Metadata leak	UUID-based	No original	Manual test:

#	Threat / Vulnerability	Feature Implemented	Defensive Logic	Validated By Test
	(original filename)	storage	filenames in database or storage	Directory listing shows only UUIDs
11	Nonce reuse (AES-GCM catastrophe)	Secure random nonces	96-bit cryptographically secure random per operation	Code review: <code>os.urandom(12)</code> per encrypt
12	Dangling shares/links after delete	Cascade delete	Foreign key <code>onDelete=CASCADE</code>	Integration test: Deleted file → shares gone
13	Malformed input (negative TTL)	Bounded TTL validation	TTL must be 1-30 days	Code review: Input validation in endpoint

Key Insight: We didn't just add features—we **eliminated attack vectors** with every design decision.

Appendix: Links & Resources

Internal Documentation:

- `README.md` — Installation, architecture, SDLC process
- `docs/threat_model.md` — Full STRIDE analysis and security requirements
- `docs/progress.md` — Development timeline and decisions
- `SECURITY.md` — Security policy and incident response
- `tests/test_security.py` — All 13 security tests

External Resources:

- STRIDE Threat Modeling: <https://cheatsheetseries.owasp.org/>
- AES-GCM: <https://csrc.nist.gov/publications/detail/sp/800-38d/final>
- OIDC/JWT: https://openid.net/specs/openid-connect-core-1_0.html
- Docker Security: <https://docs.docker.com/engine/security/>

Google Slides

Presentation: https://docs.google.com/presentation/d/1XnGRf5Vo4o3Id0oRWGWatTBEDqDfn9ZjGAW2n2Y_rT4/edit